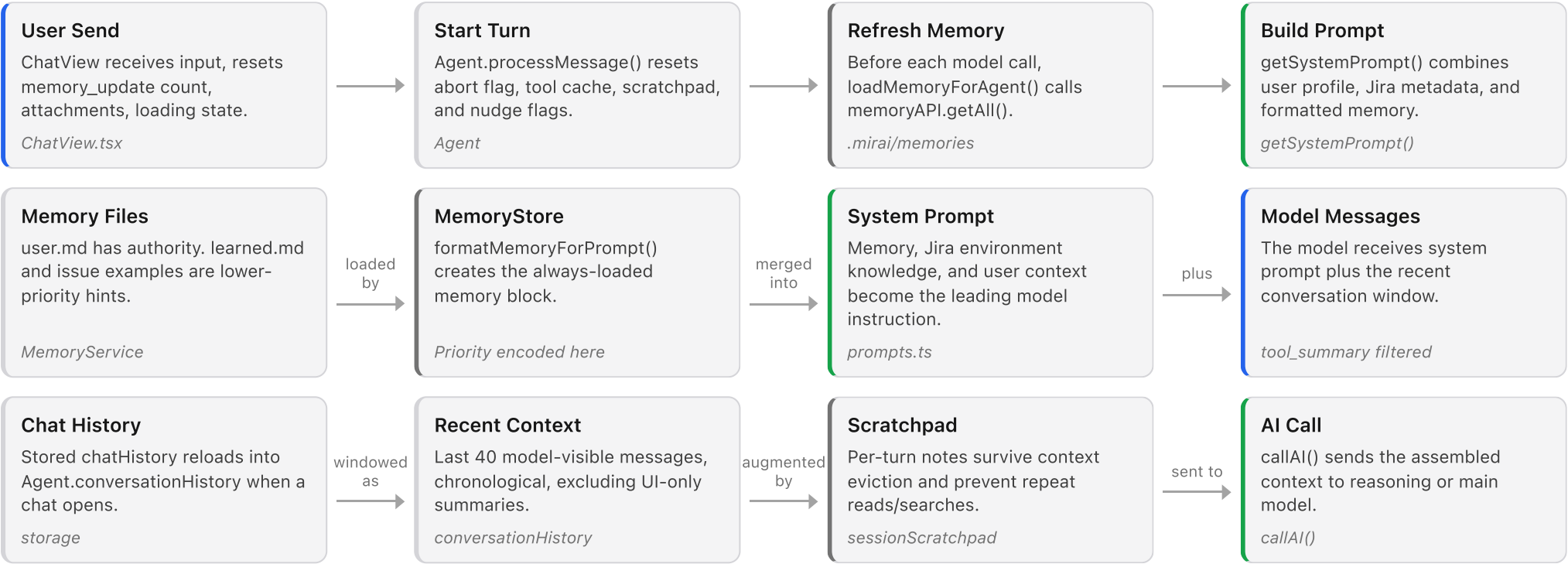


Mirai Memory and Agent Loop Flow

Two end-to-end flows: how context is assembled into each model call, and how the agent loops through tools, approval pauses, and final answers.

- context assembly
- approval branch
- loop-back paths

Flow 1: Context Assembly



Write-Back Flow

memory_update / delete

Agent requests memory changes through tools.

one update per turn



executeMemoryTool()

ChatView validates and calls memoryAPI methods.

renderer guard



MemoryService.save

Markdown files are rewritten and cache updates.

electron service



Next AI Call

loadMemory refreshes, so the saved note enters future prompts.

loop back

Flow 2: Agent Loop

runAgentLoop()

Iteration starts unless abortFlag is set or maxIterations is reached.

default cap: 10



callAI()

Refreshes memory, appends scratchpad, sends message window, streams response.

reasoning or main model



Tool calls returned?

yes: inspect tools

no: finalize



Final Answer

Action-first and think-only guards may nudge once; otherwise response is saved.

assistant message

Requires confirmation?

yes: pause

no: execute



Execute Tool

Jira, file, memory, or scratchpad tool runs immediately.

executeTool()



Record Result

Tool result enters history as [tool_result: name]; summary goes to UI only.

tool_summary filtered



Next Iteration

The model sees the result and decides whether to call more tools or answer.

back to callAI()

PendingAction

Confirmed Jira writes and attachments pause the loop for user approval.

onPendingAction()



approveAction()

Runs the deferred tool or batch, then pushes the tool result.

resume point



Scratchpad / Cache

Writes invalidate cache; useful actions append scratchpad notes.

same turn memory



runAgentLoop()

The loop resumes after approval and lets the model continue.

not a new user turn

Loop Exit Paths

- Final assistant response when no tool call remains.
- Pause on `PendingAction` until approve or reject.
- Abort boundary via `abortFlag`.
- Iteration-limit message when max loop count is reached.

Context Feedback Loop

- Tool results become future model context.
- Scratchpad notes preserve what happened this turn.
- Memory updates persist beyond the current chat.
- UI summaries remain visible to the user but hidden from the model.

Source Map

Memory files: `.mirai/memories/user.md`, `learned.md`, `issue-types/*.md` Memory service: `electron/services/memory.ts` Prompt assembly: `src/types/memory.ts` + `src/agent/prompts.ts`

Agent loop: `src/agent/index.ts` Renderer wiring: `src/components/ChatView.tsx` Tool definitions: `src/agent/tools.ts`